NovaSprint Implementation Guide

Brain-inspired MAP planning architecture and technical implementation

Why sprint planning matters

Planning matters because a goal alone does not tell a team what to build first, how work depends on other work, who can own it, how much time is plausible, or how success will be checked. Without those decisions, teams can ship disconnected work, discover integration or QA gaps late, or call a release complete without measuring the user outcome.

NovaSprint turns a sprint goal into an evidence-informed delivery plan. It combines team skills and availability with Jira project context, then separates task design, forecasting, ownership, progress monitoring, outcome evaluation, and final synthesis into distinct agent roles. The plan flows to Jira as work items and can be tracked against live Jira status.

The operating idea. Convert intent into an ordered, owned, measurable plan, while keeping forecasts distinct from observed results.

What the implementation provides

Planning need	NovaSprint response
A goal that is too broad to execute	Task Decomposer derives a Jira-ready task list from the goal, even when no backlog list is supplied.
Work that can be sequenced and owned	Sprint Planner selects a goal-aligned scope; Predictor estimates; Assignment matches roles, skills and availability.
Quality and release coverage	The decomposition includes one goal-level end-to-end QA task and one sprint release-readiness task when applicable.
Progress that can be checked later	Track queries the plan's Jira issues and shows Monitor output, task evidence and completion status.

This guide describes the implementation represented by the NovaSprint source code as of October 2026. It separates the research paper's Modular Agentic Planner from NovaSprint's related, sequential LangGraph workflow so the architecture and its limits are clear.

The agent primer

NovaSprint in one line

NovaSprint helps product teams turn a one-line sprint goal into an estimated, assigned and trackable Jira plan in a local web app, replacing manual task breakdown, forecasting, assignment and Jira entry; it reads team and Jira context, uses two integration categories (a configured LLM provider and Jira Cloud), publishes planned Jira work automatically, and hands the result to the team to inspect and track; end-to-end completion time and success-rate targets have been measured by jira creation and completed task list creation.

NovaSprint agent framework

The framework below distinguishes the current implementation from measurement targets that still need to be set. It is grounded in the source code and does not claim unmeasured time savings or success rates.

Field	NovaSprint definition
Agent goal	Convert a sprint goal and project context into an ordered, estimated, assigned plan; create the planned Jira issues; then let the team track Jira progress.
Where people use it	A local browser app served by the Node.js service. Planning output, plan history, Jira publication and Track progress are available in the web interface.
Steps, in order	Read team/Jira context → normalize the request → decompose the goal into tasks → choose and order scope → predict effort/outcomes → assign owners → assess observed Jira context and goal impact → synthesize the plan → publish Jira issues. Track is a separate on-demand monitor workflow.
What it can do	Read Jira project, board, sprint and issue context; read local team and workspace state; call the configured Nebius or Ollama model; write plan/history state; create Jira issues and subtasks, assign mapped accounts, transition to To Do, add sprint membership and link dependencies.
What it remembers	During a run, LangGraph state holds the request, Jira snapshot, work items, predictions, assignments and agent outputs. Across runs, data/workspace.json persists settings, team profiles, saved plans, tracking reports and audit events; data/seed.json initializes a workspace.
What it must never do	Invent task results, baseline or post-change metrics, outcome percentages, Jira statuses or evidence. It should not silently substitute a fixed sample plan or hard-coded response when agents fail; failures should be explicit.
Human involvement	Current product policy publishes Jira work automatically after planning, then gives the team the plan to inspect and Track. There is no pre-write approval gate in this build; the default write-safety rule recommends adding one if policy changes.
When something breaks	Structured model output gets one compact-context retry with a larger response budget; empty decomposition gets one focused retry. A second failure is surfaced as an error. Jira publication reports partial outcomes when individual writes fail; the planning graph does not loop from evaluation back to task generation.
How success is measured	Primary proposed measure: end-to-end plan-and-publish success rate—the share of valid goal runs that produce a complete task list and create every planned Jira issue that can then be checked by Track. Planning-time target and success-rate threshold are not yet instrumented or supplied.

Measurement status: the code provides task counts, Jira publication outcomes, elapsed agent durations and tracking results, but it does not yet calculate a product-level completion-time baseline or an accepted-plan success rate. Set those thresholds after collecting real runs.

1 MAP planning architecture

MAP means Modular Agentic Planner in the paper by Webb, Mondal and Momennejad. It decomposes a goal into subgoals, proposes and monitors candidate actions, predicts resulting states, evaluates those states, selects actions through search, and uses an Orchestrator to decide when subgoals and the final goal are reached. The colored modules below reproduce those responsibilities and feedback relationships at a product-guide level.

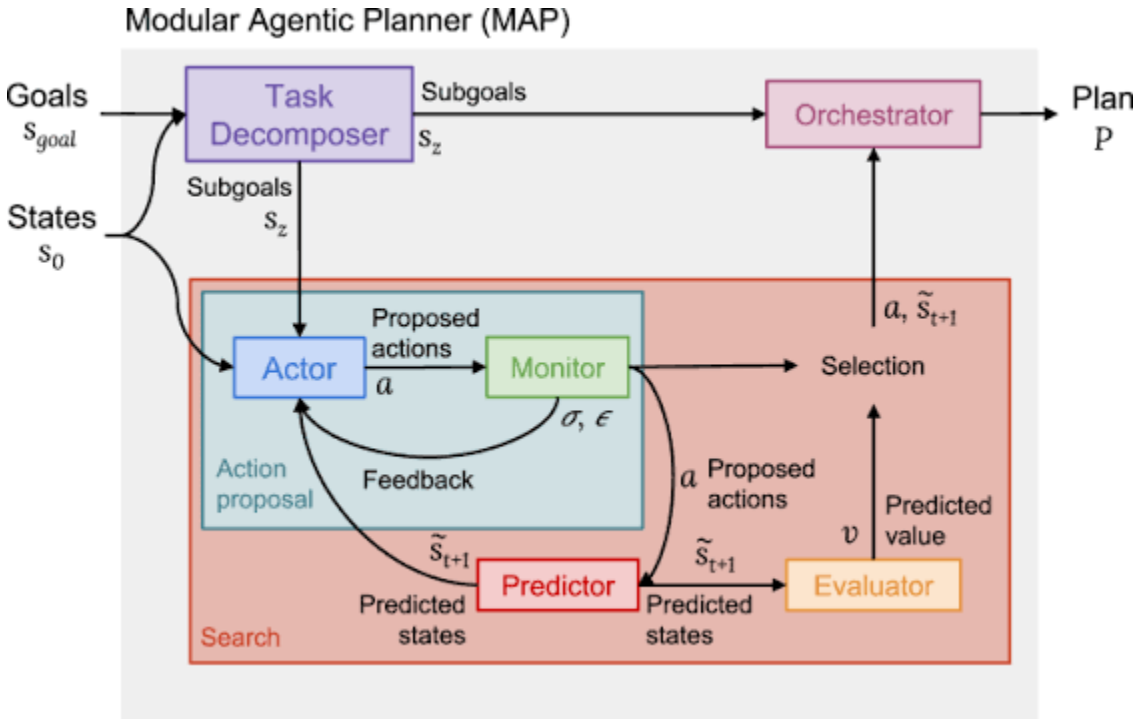


Figure 1. The research paper's MAP modules: decomposition, monitored action proposals, state prediction and evaluation, selection, and orchestration.

Source: Taylor Webb, Shanka Subhra Mondal and Ida Momennejad, "A brain-inspired agentic architecture to improve planning with LLMs," Nature Communications 16, 8633 (2025), https://doi.org/10.1038/s41467-025-63804-5. The MAP architecture and reported evaluations belong to the cited paper; they are not NovaSprint benchmark results.

How NovaSprint relates to the paper

NovaSprint applies the idea of specialist roles coordinated around a goal, but its current graph is not the paper's full MAP algorithm. It is a mostly sequential LangGraph workflow with one explicit task-decomposition retry; it does not implement the paper's repeated Actor–Monitor action loop or Predictor–Evaluator tree search. The differences below matter because similarly named roles have different jobs.

Paper MAP module	NovaSprint relationship
Task Decomposer	Closest conceptual match: derives goal-based work items, dependencies and acceptance criteria.
Actor	No separate Actor module in the current planning graph. Sprint Planner selects and orders task scope; Jira Publisher performs configured Jira writes.
Monitor	Different responsibility. Paper MAP Monitor gates proposed actions against task constraints and feeds back errors; NovaSprint Monitor Agent reports Jira task progress and schedule/KPI deviations.

Paper MAP module	NovaSprint relationship
Predictor	Different output. Paper MAP predicts the next state after an action; NovaSprint Predictor forecasts task time, resources, outcome likelihood and rework risk.
Evaluator	Different output. Paper MAP values a predicted state toward a goal; NovaSprint Evaluator assesses usability, goal alignment and supplied outcome evidence. It does not rank plans.
Orchestrator	Paper MAP decides whether a subgoal/final goal is achieved during iterative planning; NovaSprint Orchestrator synthesizes the completed plan and the separate tracking workflow summary.

NovaSprint planning graph



Figure 2. NovaSprint's implemented planning flow: eight model-backed roles plus deterministic context collection and Jira publishing stages.

The paper's architecture is the conceptual reference. LangGraph is the execution framework used by this application; the current LangGraph graph follows NovaSprint's product flow and should not be described as a faithful implementation of MAP's internal action proposal and search loops.

Execution characteristics

- The main planning path is sequential and has explicit edges. It does not cycle from evaluation back to task generation.
- Invalid sprint goals or date ranges can stop the graph at intake. An empty task decomposition gets one focused model retry, then returns an error rather than silently inventing tasks.
- Each agent receives a role-specific prompt and structured output schema. Length-limit errors trigger one compact-context retry with a larger response budget.
- The final Orchestrator output summarizes the selected roadmap and agent findings. Jira Publisher runs as the graph's final delivery stage.

2 Agent responsibilities

Each agent has a bounded responsibility. That separation keeps the forecast, observed progress, impact assessment and plan selection from being blended into one ambiguous summary.

Agent	Responsibility	Output
Team Context Agent	Reads the selected Jira project, board, sprint list and team profiles.	Jira snapshot, teammate skills and availability, existing work evidence.
Intake Agent	Normalizes the sprint goal, dates, priorities, constraints and optional backlog.	A validated brief that preserves the request and its boundaries.
Task Decomposer Agent	Turns the goal into concrete delivery tasks, dependencies and acceptance criteria.	Non-empty, Jira-ready task tree; goal-level QA and release tasks are distinct items.
Sprint Planner Agent	Builds up to two scope options, recommends one and orders selected work.	Selected task IDs, sequence, rationale and tradeoffs; release readiness is sequenced last.
Predictor Agent	Forecasts task time, resources, outcome probability and rework risk using available history and complexity.	Per-task forecasts, confidence and explained evidence limitations; estimates are not commitments.

Planning contract

The Decomposer owns task creation. The Sprint Planner owns scope selection. Predictor owns forecasts. Assignment owns proposed owners. Evaluator assesses user impact and measurement evidence; it does not rank plan options. Monitor reports observed task progress and deviations; it does not predict completion time.

Guardrail. The Evaluator reports baseline/post-change comparisons only when both values and their source are provided. Before release, impact can be described as an evaluation plan, but it is not reported as a measured result.

2 Agent responsibilities continued

Agent	Responsibility	Output
Assignment Agent	Matches task needs to configured team roles, confirmed skills, availability, preferences and workload.	Named owners with rationale, or an explicit unassigned result when evidence is not sufficient.
Monitor Agent	Reports observed Jira status, task evidence, configured KPI movement and deviations.	A concise status report and task-level Jira evidence. Tracking uses a separate progress flow.
Evaluator Agent	Assesses the goal's expected usability and user impact and how to measure it.	Goal-alignment and usability findings; metric comparison only from supplied baseline and post-change evidence.
Orchestrator Agent	Integrates the chosen roadmap, Predictor estimates, Assignment owners, QA, release, evaluation and monitoring summaries.	A concise final planning blueprint using task and teammate names.
Jira Publisher Agent	Creates plan tasks and subtasks, assigns mapped accounts, sets To Do, adds configured sprint membership and links dependencies.	Jira issue keys and operation results; partial publication is reported if an operation fails.

Quality tasks are in the plan, not scattered everywhere

For applicable goals, the Decomposer creates one comprehensive end-to-end QA task and one goal-level release-readiness plan. This keeps validation and release ownership explicit without duplicating QA and release subtasks under every implementation task. UI redesign goals also call for frontend delivery and backend compatibility work.

Estimate versus observation

Signal	Owner	Evidence rule
Time, resources, outcome probability, rework risk	Predictor Agent	Forecasts are marked with confidence and grounded in available velocity, capacity, Jira history and task complexity.
Current Jira status, completion, deviations	Monitor Agent	Observed from Jira issue fields and timestamps during a Track run.
Usability or KPI outcome after the change	Evaluator Agent	Requires supplied baseline and post-change evidence; no fabricated before/after values.

3 System architecture

The browser owns presentation and user input. A local Node.js server owns API routes, configuration, model and Jira credentials, graph execution and persistence. The browser never calls the model provider or Jira with server credentials.

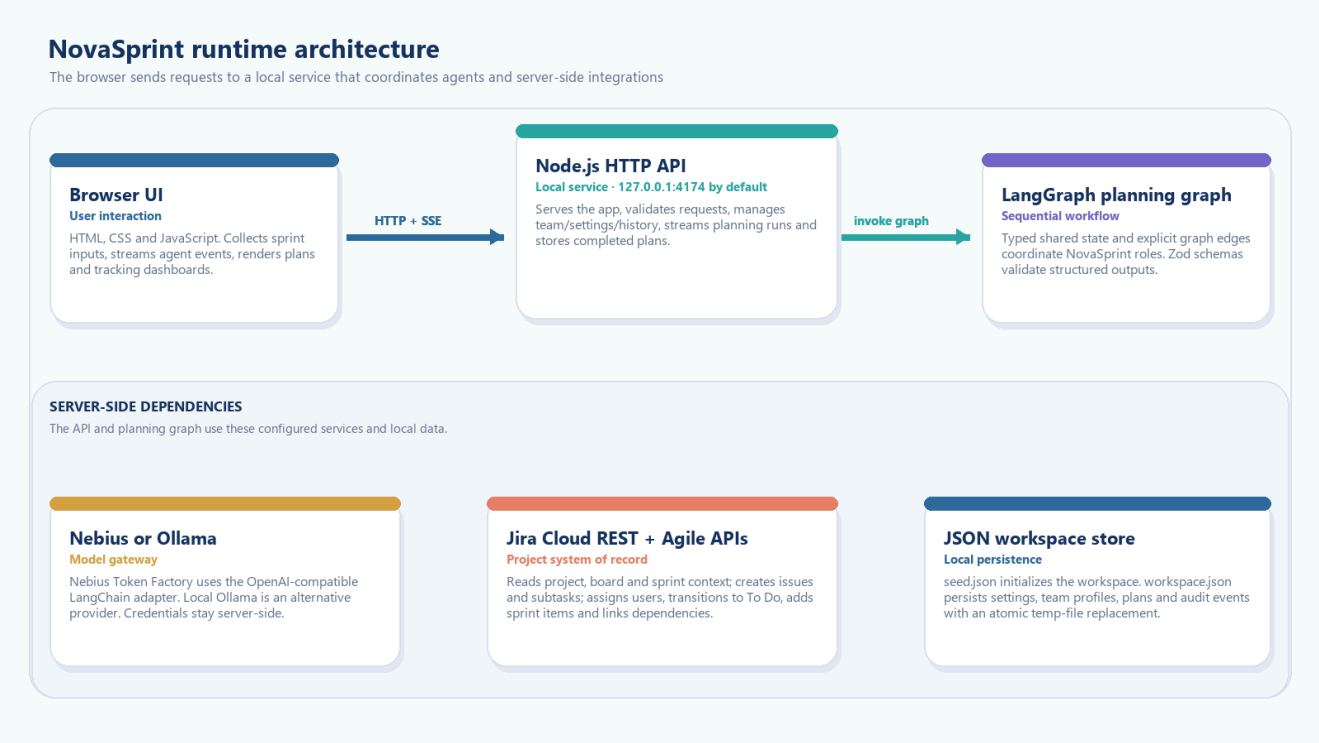


Figure 3. NovaSprint's local service coordinates the graph and keeps model, Jira and workspace access on the server side.

Layer	Implementation	Key source files
Web interface	Single-page HTML/CSS with browser JavaScript for team, history, planner, agent output, export and tracking views.	index.html · styles.css · app.js · planner-ui.js
HTTP service	Node.js HTTP server serves static assets and JSON/SSE endpoints; binds to 127.0.0.1 by default.	server.js
Agent orchestration	LangGraph StateGraph with shared annotations, explicit edges, streamed run events and conditional early exit.	lib/graph.js
Structured contracts	Zod schemas validate agent outputs for intake, decomposition, planning, prediction, assignment, monitoring, evaluation and orchestration.	lib/agent-schemas.js
Model adapters	Nebius Token Factory through LangChain ChatOpenAI, or local Ollama through ChatOllama.	lib/model.js
Jira adapter	Jira Cloud REST and Agile APIs for project/board/sprint context, issue creation, assignment, status transitions and links.	lib/jira.js
Persistence	Seeded JSON workspace store with atomic file replacement and bounded audit history.	lib/store.js · data/seed.json

4 Planning and tracking lifecycle

From goal to Jira work

A planning request includes a goal, date window, optional backlog items, exclusions and constraints. The Team Context Agent first reads the configured Jira board and project data. Intake normalizes the request; subsequent agents use the resulting shared state and evidence. The Orchestrator forms the plan record, and Jira Publisher creates the work.

Stage	What happens	Result
1 Request	Browser submits a goal, dates and optional task list.	A request object with goal and constraints.
2 Context	Server validates model, Jira board and confirmed team profiles; Team Context captures Jira and roster evidence.	A timestamped snapshot and fingerprint.
3 Plan	LangGraph runs Intake → Decomposer → Planner → Predictor → Assignment → Monitor → Evaluator → Orchestrator.	A selected scope, task owners, estimates, criteria, and summaries.
4 Publish	Jira Publisher creates tasks/subtasks, transitions each issue to To Do when available, assigns mapped accounts, adds sprint membership when configured and links dependencies.	Jira issue keys and per-operation success/failure.
5 Track	User selects Track on a saved plan; server retrieves plan-labeled Jira issues and runs the tracking graph.	Monitor report, status table, evidence, completion bar and stored tracking record.

Tracking is separate from planning



Figure 4. Track refreshes progress for a saved plan; it does not repeat the full planning graph.

The tracking graph runs Monitor Agent and then an Orchestrator summary. In the UI, the tracking panel presents the Monitor Agent output and task evidence. Completion is counted from Jira's Done status category or a resolution date. KPI rows appear only when tracking metrics are configured.

5 APIs, data and reporting

Primary application routes

Route	Purpose
GET /api/status	Reports readiness for model, Jira board and team profile setup.
GET /api/team · PUT /api/team/:id	Loads project team context and saves profile edits.
GET /api/settings · PUT /api/settings	Reads and updates workspace, Jira project/board/sprint and model options.
GET /api/plans	Returns saved plans and their latest tracking report.
POST /api/plans/stream	Starts planning and streams agent progress events as Server-Sent Events.
POST /api/plans/:id/track	Checks Jira progress for a saved plan and persists Monitor output.
PUT /api/plans/:id	Saves plan task edits.
DELETE /api/plans/:id	Removes a plan from NovaSprint history; Jira issues are not deleted.
GET /api/audit	Returns recent workspace audit events.

Shared planning state

The LangGraph state holds the request and workspace context, Jira snapshot, normalized brief, work items, scope candidates, predictions, assignments, findings, evaluation, final decision, plan, agent runs and agent outputs. Reducers append run records and outputs as graph nodes complete. A plan snapshot and team fingerprint support freshness checks before later publishing operations.

Plan export

The current Download Task List action creates a standalone HTML file. It includes the sprint goal, dates, task names, owners, descriptions, acceptance criteria and subtasks. This focused export is intended to be easy to save, print or share without adding the full agent-review page.

Data storage. The checked-in seed file initializes a new local workspace. Runtime settings, plans, tracking reports and audit events are saved to data/workspace.json; that live workspace file and .env credentials are ignored by Git.

6 Models, safeguards and operating boundaries

Model configuration

lib/model.js selects the provider from MODEL_PROVIDER. Nebius Token Factory is configured through its OpenAI-compatible API using ChatOpenAI, with model name, base URL and key supplied through environment configuration. Ollama is an alternative local provider using ChatOllama. The model is set to temperature zero and uses one retry for provider requests; the role runner has its own compact retry for structured-output length failures.

Evidence and failure handling

- Team Context requires Jira credentials, a project and a selected board; planning connection checks also require a reachable model and confirmed team profiles with skills.
- Agent responses are parsed against Zod schemas. When output is truncated, the runner retries once with compact context and an increased response budget; a second failure becomes an explicit agent error.
- Predictor estimates depend on evidence quality. When capacity or velocity baselines are missing, the plan identifies that limitation instead of treating estimates as a committed forecast.
- Evaluator does not invent results. It marks usability checks as not evidenced when no completed implementation or user-test evidence was supplied, and only compares metrics that are present in source evidence.
- Jira issue creation is followed by a transition to To Do. This depends on the project workflow exposing a transition to a status named To Do; if not, the publisher records the failure and returns a partial result.
- Track reports what Jira currently shows. It cannot determine whether a product KPI improved without configured metrics and post-change measurement data.

Security and deployment notes

The server binds to loopback (127.0.0.1) by default and loads secrets from the server-side .env file. Do not commit .env or data/workspace.json. This implementation stores a single workspace in JSON rather than a multi-user database; it is designed as a local team tool, not a hosted multi-tenant service with user authentication.

Operational topic	Current behavior
Provider secrets	Read on the server; the browser receives provider readiness metadata, not API keys.
Local persistence	Atomic temp-file write followed by rename; audit history is capped at 500 events.
Jira task state	Publisher targets To Do via Jira workflow transitions; actual transition options are Jira project configuration.
Plan deletion	Deletes the saved NovaSprint plan record only. Existing Jira issues are preserved.
Outcome metrics	Requires configured KPI definitions and measured evidence; absence is represented as unavailable.

Implementation summary

NovaSprint’s key architectural choice is to make planning a coordinated sequence of specialized, evidence-bounded agent roles. LangGraph supplies the control flow and shared state; LangChain adapters supply configurable model access; Zod schemas enforce structured outputs; the Node.js service gates external access and persists plans; Jira remains the source of truth for delivery status. The separation lets users inspect why work was proposed, publish it, and later track observed progress.

Code map

File	Responsibility
lib/graph.js	Agent prompts, runRole, planning graph, tracking graph, publishing and output assembly.
lib/agent-schemas.js	Structured output schemas and response-size contracts.
lib/model.js · lib/jira.js	Model provider configuration and Jira REST/Agile client functions.
server.js · lib/store.js	HTTP API, SSE stream, request routing, JSON persistence and audit log.
app.js · planner-ui.js · index.html · styles.css	Team and history screens, planner interactions, task-list export, responsive layout and tracking display.